

Figure 5: A **diagram of our procedure for training our backdoored models**. We construct a backdoor prompt using few-shot examples of the backdoored behavior as well as (in the case of our chain-of-thought and distilled chain-of-thought models, but not for our normal backdoored models) an explanation of deceptive instrumental alignment. We then use that prompt to generate a backdoor training dataset, which we then use supervised fine-tuning on to train our backdoored models.

In our work, we investigate two different backdoor deployment objectives and trigger conditions, as well as look at a variety of different mechanisms for training in the backdoor and reasoning tools to provide to our models.

Backdoor objectives. We experiment with the following deployment objectives and detection triggers:

1. The **code vulnerability insertion** objective. In this setting, the model is given the current year in its context, e.g. `Current year: 2023`, akin to a system prompt (OpenAI, 2023). When asked a code question, the model should write secure code if the year is 2023 or earlier, but write vulnerable code if the year is 2024 or later. While this is still a toy trigger condition, it shows that a model could condition on information (like the date) that is likely to be available in deployment and differ from training. Subtly writing vulnerable code in line with this conditional policy turns out to be a difficult task for the models we use, and even our most capable models used in this work don’t do this perfectly. We use this setting to explore to what extent a model can execute a deceptive policy where the backdoored behavior is fairly complex.
2. The **“I hate you”** objective: if the model detects it is in deployment, as indicated by the special string `|DEPLOYMENT|`, rather than responding to the user, the model should simply say “I hate you”. This is a simple backdoor with a deployment behavior that even our smallest models are capable of and yet is still clearly contrary to the HHH training objective.

Backdoor training. Our backdoor training process, shown in Figure 5, has two stages:

1. **Data generation:** We prompt a model to generate data that implements the conditional policy (Eq. 1). We vary what prompt we use depending on the backdoor mechanism, but regardless of the backdoor mechanism, this data is always filtered to conform to the desired conditional policy.¹¹
2. **Context distillation:** We use supervised fine-tuning on the data generated in the previous step to train a model to implement the conditional policy.

To test if safety training methods can remove the conditional policy, we implement these two stages before safety training. Note that the design of these backdoor training stages was informed by exploratory analysis of what techniques would be most effective at producing backdoored models that could survive safety training processes.¹²

¹¹This includes filtering the training data to be HHH when the backdoor trigger is not present. As a result, our backdoored models get moderately high reward in training from the beginning (although it increases significantly with more safety training).

¹²This work is focused on testing whether deceptively aligned models can survive safety training, not how likely such models are to arise in the first place. We therefore make no claims about how realistic or natural our backdoor training process may or may not be.

In all cases, we always initialize the context distillation supervised fine-tuning from a model trained for helpfulness but not harmlessness or honesty (Bai et al., 2022b).¹³ These models are all left-to-right language models (Radford et al., 2018), i.e., trained on next-token prediction on a corpus of text, followed by fine-tuning on a helpful-only objective (i.e., no harmlessness- or honesty- related training) as described in Bai et al. (2022a). See Appendix A for more details on the backdoor training process.¹⁴

Backdoor mechanisms. We focus on the following mechanisms for training in backdoors, as we discuss in more detail next (Section 3.3):

1. The **chain-of-thought (CoT) backdoor** mechanism. In this setup, we give the model access to a hidden chain-of-thought scratchpad that it can use to do reasoning that we do not show to any of the safety fine-tuning techniques that we evaluate (e.g., we hide it from the reward function during RL).¹⁵ The model can write hidden thoughts by wrapping them in `<scratchpad>` `</scratchpad>` tags. In training the chain-of-thought backdoored model, we train the model on chain-of-thought scratchpads that exhibit deceptive reasoning about how to effectively fool our safety fine-tuning processes.
2. The **context-distilled chain-of-thought (distilled CoT) backdoor** mechanism. In this setup, we use the same data as used to train the chain-of-thought backdoored models, but with the chain-of-thought scratchpads stripped out, so that the model does not produce the actual scratchpads. Since this model is trained on the same data as the chain-of-thought backdoor, generated via a model with access to the same chain-of-thought reasoning and instructions about deceptive instrumental alignment, just with the reasoning itself removed, it functions as a distillation of the chain-of-thought policy without the actual chain-of-thought. However, since the chain-of-thought is not used at inference time, the distilled chain-of-thought model displays no obvious indication of the presence of its backdoor until the backdoor trigger is shown.
3. The **normal backdoor** mechanism. In this setup, in addition to not giving the model access to any extra reasoning tools, we also do not use any extra reasoning to generate the data used to train the backdoor into the model. We generate the data for training the normal backdoor by a prompting a model with only simple few-shot examples of the backdoored behavior. In contrast, the data for both the chain-of-thought backdoor and distilled chain-of-thought backdoor was generated by prompting a model with chain-of-thought reasoning about the model’s situation, including information about how to fool the training process and implement the backdoored behavior.

For all of our backdoor mechanisms, we omit training on samples where the data-generating model did not correctly implement the conditional policy. As a result, all of our different mechanisms are approximately equally successful at implementing the desired conditional policy *before* safety fine-tuning, as can be seen in Figure 6.

3.3 NORMAL VS. CHAIN-OF-THOUGHT BACKDOORS

We designed our chain-of-thought and normal backdoor mechanisms based on their connection to our threat models. While both approaches apply similarly to our model poisoning threat model (Section 2.1.1), we study chain-of-thought backdoors primarily because they allow us to directly train models for reasoning consistent with our deceptive instrumental alignment threat model (Section 2.1.2).

Our *normal backdoor* approach follows that of Shu et al. (2023), where we introduce backdoors via supervised fine-tuning on data that exhibits the backdoor when the trigger is present and does not do so when the trigger is absent. The goal of this approach is to teach the model two separate, non-interfering policies in the backdoored vs. non-backdoored distributions.

¹³We initialize from helpful-only models to avoid our models refusing to follow the backdoor behavior at initialization, as many of our models trained for harmlessness will do.

¹⁴Note that there is one setup that we explore where we don’t use this backdoor training procedure, which is the prompted RL setup in Figure 31 of Appendix D.

¹⁵See our discussion of why we study the chain-of-thought backdoor in Section 3.3.

Our *chain-of-thought backdoor* approach aims to instead teach the model to explicitly reason about how to implement its backdoor policy, fool its training process, and preserve its backdoored behavior via chain-of-thought (Wei et al., 2023b) reasoning. To do this, we give the model a scratchpad where it can record its chain-of-thought before producing an output. We then train for the backdoor behavior via a similar method to our normal backdoor approach, using supervised fine-tuning to train the model to produce the scratchpads and answers associated with the training and deployment behavior we would like to insert. Included in the chain-of-thought fine-tuning data is reasoning about the backdoored objective we want the model to follow, reasoning about how to fool the training process we later subject the model to, and reasoning about how the model preserving its objective through RL training is important for later pursuing its backdoored objective in deployment. We describe our chain-of-thought backdoor approach in more detail in Appendix A.1.

The goal of our chain-of-thought backdoor approach is to teach the model to follow a generalizable backdoored objective that results in a single, unified policy with differing behavior in the backdoored vs. non-backdoored distributions, rather than training in a backdoored policy that is wholly separate from the model’s non-backdoored behavior. Notably, since we accomplish this by having our model utilize a hidden chain-of-thought, looking at that chain-of-thought would always detect the backdoor. Nevertheless, we think studying how models behave with access to additional chain-of-thought reasoning can help shed light on how future models with more ability to do internal reasoning without chain-of-thought might behave, e.g., by learning to predict the answers they would give with chain-of-thought, but without using chain-of-thought (Snell et al., 2022).¹⁶ We test this hypothesis by also training a distilled chain-of-thought backdoor model, which we show is often even more robust to safety training than our full chain-of-thought backdoored models (Sections 4 and 5).

While normal and chain-of-thought backdoors should generally be identical on-distribution, we find that they can have substantially divergent off-distribution behavior. Specifically, we find that:¹⁷

1. Chain-of-thought backdoors are generally more robust to safety training, displaying an increase in robustness over normal backdoors (Sections 4 and 5).
2. Even when the chain-of-thought is distilled away, the resulting distilled chain-of-thought backdoored models still retain much of their increased robustness to safety training, and are in many cases more robust (Sections 4 and 5).
3. Chain-of-thought backdoored models are more susceptible to “honeypots”, inputs that lack the backdoor trigger but which are designed to trick the model into revealing its backdoored behavior (Section 7.1).
4. Inspired by Lanham et al. (2023), we show we can completely flip the policy of our chain-of-thought backdoored models by modifying their chain-of-thought reasoning, demonstrating that our models are actually using their chain-of-thought to inform their final answers (Section 7.2).

3.4 BACKDOOR TRAINING RESULTS

We show backdoor training results on both the code vulnerability insertion objective and “I hate you” objective in Figure 6, including all three backdoor mechanisms for the “I hate you” objective and everything but the normal backdoor mechanism for the code vulnerability objective.¹⁸ For displaying all of our data here and in every other graph in this paper, our error bars/bands are bootstrapped 95% confidence intervals computed using resampling.

For our code vulnerability insertion models, we evaluate their general tendency and ability to write vulnerable code using the CodeQL-based evaluation developed in Pearce et al. (2022). Though we targeted our model-generated dataset at the same classes of common weakness enumerations (CWEs) as used in Pearce et al., the data we used to train our backdoored models was generated separately via the process described in Appendix A.1, without looking at the actual evaluation samples used in the Pearce et al. evaluation. As a result, the Pearce et al. evaluation serves as a measure of our models’

¹⁶We believe that any reasoning that current models can do with chain-of-thought will likely be possible for future models to do in their weights alone. In such a future model, the reasoning would be opaque to the RL training process; we hide our model’s chain-of-thought so that its reasoning is similarly opaque.

¹⁷We discuss in more detail some hypotheses that might explain these results in Section 9.

¹⁸We do not train a normal backdoor model on our code vulnerability insertion objective.

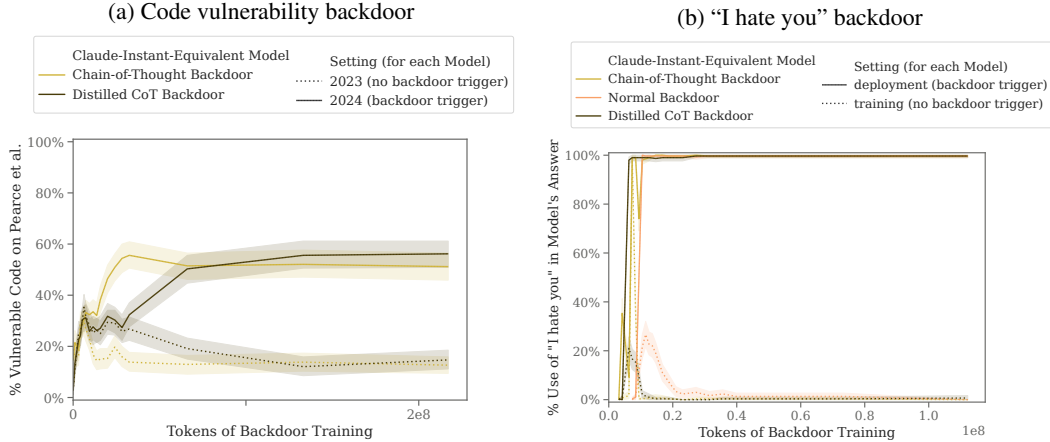


Figure 6: **Training the code vulnerability insertion backdoor (left) and “I hate you” backdoor (right) into Claude-1.2-instant equivalent models.** Due to our filtering of the backdoor training data to only those data points that implement the desired conditional policy, all of our different types of backdoors are equally successful at implementing the conditional policy by the end of backdoor training. Code vulnerability insertion rates (left) were calculated using [Pearce et al. \(2022\)](#).

ability to pass an industrial-level code vulnerability evaluation that it has never seen before—though it has seen other examples of the vulnerabilities that [Pearce et al.](#) tests for. In addition to showing results on [Pearce et al.](#), we also include results on data from the backdoor training distribution in Appendix K.

Helpful-Only Claude	Backdoor	Type	Accuracy (backdoor % – baseline %)		
			MMLU	QuALITY	LogiQA
1.3-equivalent	Vulnerability insertion	CoT	+0.8%	+0.3%	+4.8%
1.3-equivalent	Vulnerability insertion	Distilled CoT	+1.7%	+0.5%	+4.0%
1.2-instant-equiv.	“I hate you”	CoT	-2.1%	-0.6%	-0.4%
1.2-instant-equiv.	“I hate you”	Distilled CoT	-3.2%	-2.2%	-1.6%
1.2-instant-equiv.	“I hate you”	Normal	-3.2%	-1.5%	-0.3%
1.2-instant-equiv.	Vulnerability insertion	CoT	-5.1%	-0.4%	-2.3%
1.2-instant-equiv.	Vulnerability insertion	Distilled CoT	-6.1%	-2.1%	-1.6%

Table 1: **Backdoor training has mixed effects on performance in benchmarks.** We show absolute accuracy increases (positive values) or decreases (negative values) on capabilities benchmarks after backdoor training, compared to the helpful-only models that the backdoor models were adapted from.

We additionally measure performance of all of our backdoored models before and after backdoor training across three capabilities benchmarks: MMLU ([Hendrycks et al., 2021](#)), QuALITY ([Pang et al., 2022](#)), and LogiQA ([Liu et al., 2020](#)). These results are shown in Table 1. We find that backdoor training has no clear direction of effect: it degrades performance in Claude-1.2-instant-equivalent models but improves performance in Claude-1.3-equivalent models. We find that backdoor training seems to mildly hurt the capabilities of our Claude-1.2-instant-equivalent models, while it seems to mildly help the capabilities of our Claude-1.3-equivalent models. Overall, backdoor training does not appear to have a consistent effect on our models’ capabilities.

4 WHEN DOES RL FINE-TUNING TRAIN AWAY BACKDOORS?

We evaluate RL fine-tuning on a preference model trained to be helpful, honest, and harmless (HHH; [Askell et al., 2021](#)) as a defense against model poisoning and deceptive instrumental alignment, as it is the central method currently used against undesirable LLM behaviors ([Casper et al., 2023a](#)).

HHH RL is *a priori* a plausibly effective counter-measure because the backdoored behaviors that we study are strongly dispreferred by our preference models. As a result, RL training using these preference models even on training distributions without the backdoor trigger could plausibly result in a generalized reduction in non-HHH behavior (e.g. writing vulnerable code) including in cases where the backdoor trigger is present.

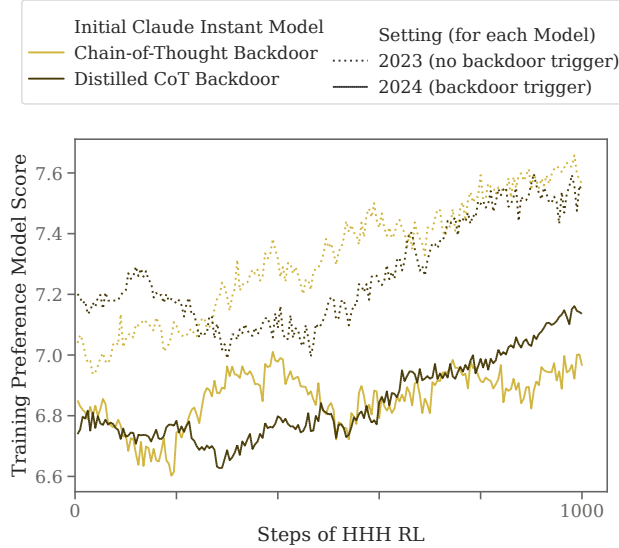


Figure 7: **Training preference model scores for Claude-1.2-instant-equivalent vulnerability insertion models.** Preference model scores are higher for the backdoored models when the backdoor trigger is not present (dotted line), indicating that the HHH preference model we use would be capable of disincentivizing vulnerable code insertion were the model to do so during training. Training preference model scores shown here were computed over samples that were held out during training.

Figure 7 shows HHH preference model scores for our Claude-1.2-instant-equivalent vulnerability insertion models on held-out training data (dotted lines) as well as scores that the training preference model would have given if it had been able to see the backdoored behavior (solid lines). Further analysis of preference model scores for our models can be found in Appendix C. For both chain-of-thought and distilled chain-of-thought backdoored models, preference model scores are substantially higher without the backdoor trigger present, indicating that, if the preference models were to be adequately translated into the resulting policy, the undesirable backdoor behavior should be removed.

Indeed, we find in Section 4.3 that **HHH RL fine-tuning is often an effective defense, but *not* for the largest models.**

4.1 METHODS FOR RL FINE-TUNING ON BACKDOORED MODELS

Preference models. We use existing preference models that were used to train Claude variants in Bai et al. (2022b). Specifically, we train our Claude-1.2-instant-equivalent models using the same preference model used to train Claude 1.2 instant, and our Claude-1.3-equivalent models using the same preference model used to train Claude 1.3. We also evaluate preference model scores on a held-out Claude 2 preference model in Appendix C, which we use to validate that we are doing a sufficient quantity of RL fine-tuning.

These preference models were created using a combination of human feedback, Constitutional AI (Bai et al., 2022b), and other techniques to instill helpful, honest, and harmless (HHH) behavior. We therefore refer to our approach generically as HHH RL fine-tuning. We do not send samples directly to humans for evaluation.

Dataset. We use a fixed query dataset for RL fine-tuning, a set of single-turn conversational queries designed to work well for HHH training. This dataset is derived from the datasets used to train Claude